AI Practical Viva Notes

Original practical questions and code from your journal, with short explanations and viva answers.

PRACTICAL 1-A: Traffic Signal

Write a Python program to decide what a vehicle should do based on the traffic signal.

CODE FROM JOURNAL

```
table = {
   "Red": "Stop",
   "Yellow": "Wait",
   "Green": "Go"
}

signal = input("Enter Signal: ")
print("Vehicle should:", table.get(signal, "Invalid Signal"))
```

Explanation: Uses a dictionary to match a signal colour with the correct action.

Possible Viva Questions

Q. What is a dictionary?
A. A Python data structure that stores key-value pairs.

Q. What does input() do?
A. Takes input from the user.

Q. What does table.get() do?
A. Returns the value for a key, or the default value if the key is missing.

Q. What is the output for Red?
A. Vehicle should: Stop.

PRACTICAL 1-B: Environment Properties

Identify the properties of Chess and Self-Driving Car environments.

CODE FROM JOURNAL

```
environment = input("Enter Environment (Chess / Self Driving Car): ")
if environment.lower() == "chess":
   print("Fully Observable")
   print("Deterministic")
   print("Sequential")

elif environment.lower() == "self driving car":
   print("Partially Observable")
   print("Stochastic")
   print("Sequential")
else:
   print("Environment not found")
```

Explanation: The program displays the environment properties based on the user's choice.

Possible Viva Questions

Q. What is fully observable?
A. The agent can access all relevant information about the state.

Q. What is partially observable?
A. The agent cannot access all relevant information.

Q. What is deterministic?
A. An action has a predictable result.

Q. What is stochastic?
A. The result involves uncertainty.

Q. What does lower() do?
A. Converts text to lowercase.

PRACTICAL 2-A: 8-Puzzle using BFS

Find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).

CODE FROM JOURNAL

```
from collections import deque

def bfs(s, g):
   q = deque([(s, [])])
   v = {s}
   while q:
      s, p = q.popleft()
      if s == g:
         return p
      i = s.index(0)
      for d in [-3, 3, -1, 1]:
         j = i + d
         if 0 <= j < 9:
            if (d == -1 and i % 3 == 0) or (d == 1 and i % 3 == 2):
               continue
            t = list(s)
            t[i], t[j] = t[j], t[i]
            t = tuple(t)
            if t not in v:
               v.add(t)
               q.append((t, p + [t]))

start = (1,2,3,4,0,6,7,5,8)
goal = (1,2,3,4,5,6,7,8,0)
for x in bfs(start, goal):
   print(x)
```

Explanation: BFS explores states level by level. Zero represents the blank tile. The queue stores states waiting to be explored.

Possible Viva Questions

Q. What is BFS?
A. Breadth-First Search explores level by level.

Q. Which data structure is used?
A. Queue.

Q. What is FIFO?
A. First In, First Out.

Q. Why use deque?
A. It provides efficient queue operations.
Q. Why use visited set v?
A. To avoid exploring the same state repeatedly.

PRACTICAL 2-B: 2-Jug Problem using DFS

Solve the 2-jug problem using Depth-First Search (DFS).

CODE FROM JOURNAL

```
visited = set()

def dfs(a, b):
   if (a, b) in visited:
      return False
   visited.add((a, b))
   print((a, b))
   if a == 2 or b == 2:
      print("Goal Reached!")
      return True
   moves = [
      (4, b),            # Fill 4L jug
      (a, 3),            # Fill 3L jug
      (0, b),            # Empty 4L jug
      (a, 0),            # Empty 3L jug
      (max(0, a-(3-b)), min(3, b+a)), # Pour 4L -> 3L
      (min(4, a+b), max(0, b-(4-a))) # Pour 3L -> 4L
   ]
   for x, y in moves:
      if dfs(x, y):
         return True
   return False

dfs(0, 0)
```

Explanation: DFS explores one path deeply before trying another. The pair (a, b) shows the water levels in the 4-litre and 3-litre jugs.

Possible Viva Questions

Q. What is DFS?
A. Depth-First Search explores a branch deeply before backtracking.
Q. Which data structure is used?
A. A stack or recursion.
Q. What is the goal state?
A. A state where either jug contains 2 litres.
Q. Why use visited?
A. To avoid revisiting states.
Q. What is backtracking?
A. Returning to an earlier state to try another path.

PRACTICAL 2-C: Uniform Cost Search

Find the least-cost path from Arad to Bucharest using Uniform Cost Search (UCS).

CODE FROM JOURNAL

```
from heapq import heappush, heappop

print("ayesha siddiqui ")
graph = {
   'Arad': [('Sibiu',140), ('Timisoara',118), ('Zerind',75)],
   'Zerind': [('Oradea',71)],
   'Oradea': [('Sibiu',151)],
   'Timisoara': [('Lugoj',111)],
   'Lugoj': [('Mehadia',70)],
   'Mehadia': [('Drobeta',75)],
   'Drobeta': [('Craiova',120)],
   'Craiova': [('Pitesti',138)],
   'Sibiu': [('Fagaras',99), ('Rimnicu',80)],
   'Fagaras': [('Bucharest',211)],
   'Rimnicu': [('Pitesti',97)],
   'Pitesti': [('Bucharest',101)],
   'Bucharest': []
}

pq = []
heappush(pq, (0, 'Arad', ['Arad']))
visited = set()

while pq:
   cost, city, path = heappop(pq)
   if city in visited:
      continue
   visited.add(city)
   if city == "Bucharest":
      print("Path:", path)
      print("Cost:", cost)
      break
   for next_city, distance in graph[city]:
      if next_city not in visited:
         heappush(pq, (cost + distance, next_city, path + [next_city]))
```

Explanation: UCS uses a priority queue and expands the path with the lowest total cost first.

Possible Viva Questions

Q. What is UCS?
A. An uninformed search algorithm that expands the lowest-cost path first.

Q. Which data structure is used?
A. Priority queue.

Q. What is heappush()?
A. Adds an item to the priority queue.

Q. What is heappop()?
A. Removes the item with the smallest priority.

Q. What is path cost?
A. The sum of the edge costs along a path.

PRACTICAL 3-A: Greedy Best-First Search

Solve the 8-puzzle using Greedy Best-First Search with the Manhattan Distance heuristic.

CODE FROM JOURNAL

```
from queue import PriorityQueue

goal = [[1,2,3],[4,5,6],[7,8,0]]
start = [[1,2,3],[4,0,6],[7,5,8]]

def h(s):
   d = 0
   for i in range(3):
      for j in range(3):
         if s[i][j]:
            x, y = divmod(s[i][j]-1, 3)
            d += abs(i-x) + abs(j-y)
   return d

def swap(s, x, y, a, b):
   t = [r[:] for r in s]
   t[x][y], t[a][b] = t[a][b], t[x][y]
   return t

q = PriorityQueue()
q.put((h(start), start))

while not q.empty():
   _, s = q.get()
   print(*s, sep="\n")
   print()
   if s == goal:
      print("Goal Reached")
      break
   for i in range(3):
      for j in range(3):
         if s[i][j] == 0:
            x, y = i, j
   for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
      a, b = x+dx, y+dy
      if 0 <= a < 3 and 0 <= b < 3:
         q.put((h(swap(s,x,y,a,b)), swap(s,x,y,a,b)))
```

Explanation: Greedy Best-First Search selects a state using h(s), the heuristic estimate. Manhattan Distance adds each tile's horizontal and vertical distance from its goal position.

Possible Viva Questions

Q. What is a heuristic?
A. An estimate of the distance or cost to the goal.

Q. What is Manhattan Distance?
A. The sum of horizontal and vertical distances of tiles from their goal positions.

Q. What does PriorityQueue do?
A. Removes the item with the smallest priority first.

Q. Is Greedy Search always optimal?
A. No, it does not always find the least-cost solution.

PRACTICAL 3-B: A* Search for 8-Puzzle

Find the shortest path using A* Search with the Manhattan Distance heuristic.

CODE FROM JOURNAL

```
from queue import PriorityQueue

goal = [[1,2,3],[4,5,6],[7,8,0]]
start = [[1,2,3],[4,0,6],[7,5,8]]
print("AYESHA SIDDIQUI ")

def h(s):
   d = 0
   for i in range(3):
      for j in range(3):
         if s[i][j]:
            x, y = divmod(s[i][j]-1, 3)
            d += abs(i-x) + abs(j-y)
   return d

def swap(s,x,y,a,b):
   t = [r[:] for r in s]
   t[x][y], t[a][b] = t[a][b], t[x][y]
   return t

q = PriorityQueue()
q.put((h(start),0,start))
visited = []

while not q.empty():
   f,g,s = q.get()
   if s in visited:
      continue
   visited.append(s)
   print(*s, sep="\n")
   print()
   if s == goal:
      print("Goal Reached")
      break
   for i in range(3):
      for j in range(3):
         if s[i][j] == 0:
            x,y = i,j
   for dx,dy in [(-1,0),(1,0),(0,-1),(0,1)]:
      a,b = x+dx,y+dy
      if 0<=a<3 and 0<=b<3:
         n = swap(s,x,y,a,b)
         if n not in visited:
            q.put((g+1+h(n), g+1, n))
```

Explanation: A* uses f(n) = g(n) + h(n), combining the cost already travelled with the estimated remaining cost.

Possible Viva Questions

Q. What is the A* formula?
A. f(n) = g(n) + h(n).

Q. What is g(n)?
A. The actual cost from the start to the current state.

Q. What is h(n)?
A. The estimated cost from the current state to the goal.

Q. How does A* differ from Greedy Search?
A. A* uses g+h; Greedy Search uses only h.

PRACTICAL 3-C: A* Search from Arad to Bucharest

Find the shortest path from Arad to Bucharest using A* with straight-line distance as the heuristic.

CODE FROM JOURNAL

```
from queue import PriorityQueue

print("AYESHA SIDDIQUI ")
graph = {
   'Arad':[('Sibiu',140),('Timisoara',118),('Zerind',75)],
   'Sibiu':[('Fagaras',99),('Rimnicu',80)],
   'Fagaras':[('Bucharest',211)],
   'Rimnicu':[('Pitesti',97)],
   'Pitesti':[('Bucharest',101)],
   'Timisoara':[],
   'Zerind':[],
   'Bucharest':[]
}

h = {
   'Arad':366, 'Sibiu':253, 'Fagaras':176,
   'Rimnicu':193, 'Pitesti':100, 'Timisoara':329,
   'Zerind':374, 'Bucharest':0
}

q = PriorityQueue()
q.put((366,0,'Arad',['Arad']))
visited = []

while not q.empty():
   f,g,node,path = q.get()
   if node in visited:
      continue
   visited.append(node)
   if node == 'Bucharest':
      print("Path:", " -> ".join(path))
      print("Cost:", g)
      break
   for n,c in graph[node]:
      if n not in visited:
         q.put((g+c+h[n], g+c, n, path+[n]))
```

Explanation: This program uses road distances as costs and straight-line estimates as the heuristic.

Possible Viva Questions

Q. What is the goal node?
A. Bucharest.

Q. What does h(node) mean?
A. Estimated remaining distance to Bucharest.

Q. Why use A*?
A. It combines actual cost and estimated remaining cost.

Q. What does path store?
A. The sequence of cities visited.

PRACTICAL 4-A: Bayes' Rule

Compute the probability that a person has a disease given a positive test result.

CODE FROM JOURNAL

```
P_Disease = 0.01
P_NoDisease = 1 - P_Disease

P_Positive_Disease = 0.99
P_Positive_NoDisease = 0.05

P_Positive = (P_Positive_Disease * P_Disease) + \
        (P_Positive_NoDisease * P_NoDisease)

P_Disease_Positive = (P_Positive_Disease * P_Disease) / P_Positive

print("----- Bayes' Rule: Medical Test -----")
print("Probability of Disease:", P_Disease)
print("Probability of Positive Test:", P_Positive)
print("Probability of Disease given Positive Test:",
    P_Disease_Positive)

print("Percentage:",
    round(P_Disease_Positive * 100, 2), "%")
```

Explanation: Bayes' Rule updates the probability of disease after considering a positive test result.

Possible Viva Questions

Q. What is Bayes' Rule?

A. A formula for calculating conditional probability.

Q. What is prior probability?

A. The probability before considering new evidence.

Q. What is posterior probability?

A. The updated probability after considering evidence.

Q. What is a false positive?

A. A positive test result when the person does not have the disease.

PRACTICAL 5-A: Naive Bayes Classification

Implement a Naive Bayes classifier using a small weather dataset.

CODE FROM JOURNAL

```
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X = [
  ['Sunny','Hot','High'],
  ['Sunny','Hot','High'],
  ['Overcast','Hot','High'],
  ['Rain','Mild','Normal'],
  ['Rain','Cool','Normal'],
  ['Rain','Cool','High'],
  ['Overcast','Cool','Normal'],
  ['Sunny','Mild','High']
]
y = ['No','No','Yes','Yes','Yes','No','Yes','No']

le = LabelEncoder()
```

```
X = [le.fit_transform(row) for row in X]
y = le.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.25, random_state=1
)

model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Predicted:", pred)
print("Accuracy:", accuracy_score(y_test, pred) * 100, "%")
```

Explanation: Naive Bayes is a probabilistic classifier based on Bayes' theorem and assumes features are conditionally independent given the class.

Possible Viva Questions

Q. What is Naive Bayes?
A. A probability-based classification algorithm.

Q. Why is it called naive?
A. It assumes features are conditionally independent.

Q. What does LabelEncoder do?
A. Converts category labels into numbers.

Q. What is accuracy?
A. The proportion of predictions that are correct.

PRACTICAL 6-A: Bayesian Network

Create a Bayesian Network using the Burglary-Alarm example.

CODE FROM JOURNAL

```
from pgmpy.models import DiscreteBayesianNetwork

model = DiscreteBayesianNetwork([
   ('Burglary', 'Alarm'),
   ('Earthquake', 'Alarm'),
   ('Alarm', 'JohnCalls'),
   ('Alarm', 'MaryCalls')
])

print("Bayesian Network:")
print(model.edges())
```

Explanation: A Bayesian Network represents variables and their relationships as a directed graph.

Possible Viva Questions

Q. What is a Bayesian Network?
A. A graphical model representing probabilistic dependencies.

Q. What does a node represent?
A. A random variable.

Q. What does an edge represent?
A. A dependency between variables.

Q. What is pgmpy?
A. A Python library for probabilistic graphical models.

PRACTICAL 6-B: Posterior Probability

Compute posterior probability using a Bayesian Network: Burglary given Alarm = True.

CODE FROM JOURNAL

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
   ('Burglary', 'Alarm'),
   ('Earthquake', 'Alarm')
])

cpd_b = TabularCPD('Burglary', 2, [[0.999], [0.001]])
cpd_e = TabularCPD('Earthquake', 2, [[0.998], [0.002]])

cpd_a = TabularCPD(
   'Alarm', 2,
   [[0.999, 0.06, 0.71, 0.05],
    [0.001, 0.94, 0.29, 0.95]],
   evidence=['Burglary', 'Earthquake'],
   evidence_card=[2, 2]
)

model.add_cpds(cpd_b, cpd_e, cpd_a)
inference = VariableElimination(model)
result = inference.query(
   variables=['Burglary'],
   evidence={'Alarm': 1}
)
print(result)
```

Explanation: The program uses conditional probability tables and variable elimination to estimate the probability of burglary when the alarm is true.

Possible Viva Questions

Q. What is a CPT/CPD?
A. A table describing a variable's probabilities given its parent variables.

Q. What is posterior probability?
A. Probability updated using evidence.

Q. What is Variable Elimination?
A. An inference method that calculates probabilities by eliminating hidden variables.

Q. What is evidence here?
A. Alarm = True.

PRACTICAL 7-B: Linear Regression

Implement Linear Regression using a suitable regression dataset.

CODE FROM JOURNAL

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
df = pd.read_csv("student_scores.csv")
print(df.head())

X = df[["Hours"]]
y = df["Scores"]

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

Explanation: Linear Regression predicts a continuous value. This practical uses study hours to predict scores. The CSV file must contain Hours and Scores columns.

Possible Viva Questions

Q. What is Linear Regression?
A. A supervised learning method that predicts continuous numerical values.

Q. What is X?
A. The input feature, Hours.

Q. What is y?
A. The target value, Scores.

Q. What is MSE?
A. Mean Squared Error, the average squared prediction error.

Q. What is R2 score?
A. A measure of how well the model explains variation in the target.

PRACTICAL 7-C: MNIST Binary Classifier

Using MNIST, implement a binary classifier (digit versus not digit). The journal code labels digit 5 as the positive class.

CODE FROM JOURNAL

```
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

mnist = fetch_openml("mnist_784", version=1, as_frame=False)
X = mnist.data
y = mnist.target.astype(int)

y_binary = (y == 5)

X_train, X_test, y_train, y_test = train_test_split(
   X, y_binary, test_size=0.2, random_state=42
)
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Training Accuracy:", model.score(X_train, y_train))
print("Testing Accuracy:", accuracy)
```

Explanation: MNIST contains handwritten digit images. This code classifies images as digit 5 or not digit 5. An internet connection may be needed to download MNIST.

Possible Viva Questions

Q. What is MNIST?
A. A dataset of handwritten digit images from 0 to 9.

Q. What is binary classification?
A. Classification into two classes.

Q. What does y == 5 do?
A. Marks digit 5 as True and all other digits as False.

Q. What is Logistic Regression used for?
A. A classification algorithm that estimates class probabilities.

PRACTICAL 7-D: K-Fold Cross-Validation

Implement K-Fold Cross-Validation on a classification model.

CODE FROM JOURNAL

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import KFold, cross_val_score

iris = load_iris()
X = iris.data
y = iris.target

model = LogisticRegression(max_iter=200)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kf)

print("Accuracy for each fold:", scores)
print("Average Accuracy:", scores.mean())
```

Explanation: K-Fold splits data into five folds and evaluates the model using each fold in turn.

Possible Viva Questions

Q. What is cross-validation?
A. A method for evaluating a model on multiple data splits.

Q. What does K mean?
A. The number of folds.

Q. What does n_splits=5 mean?
A. The data is divided into five folds.

Q. Why use cross-validation?
A. To get a more reliable estimate of model performance.

PRACTICAL 7-E: Decision Tree Classifier

Implement a Decision Tree classifier using the Iris dataset.

CODE FROM JOURNAL

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Training Accuracy:", model.score(X_train, y_train))
print("Testing Accuracy:", accuracy)
```

Explanation: A Decision Tree learns decision rules from training data and predicts the Iris flower class.

Possible Viva Questions

Q. What is a Decision Tree?
A. A supervised learning algorithm that uses a tree of decision rules.

Q. What is a root node?
A. The first node in the tree.

Q. What is a leaf node?
A. A final node that gives a prediction.

Q. What is the Iris dataset?
A. A dataset of measurements for three iris flower species.

Q. What is overfitting?
A. When a model fits training data too closely and performs worse on unseen data.

Important note

The selected practicals above follow the practicals present in your uploaded journal. Your journal does not include a Jira practical in its 26 pages. The source code is kept faithful to the journal, with indentation formatted for readability. The original journal uses external files student_scores.csv for Linear Regression and downloads MNIST for the MNIST practical.